Evaluating LLMs as Live Strategic Agents:
Provider Ranking, Hybrid Decomposition, and Operational Gaps in Timed Risk Play

H. C. Ekne

May 10, 2026

Abstract

Static benchmark scores are an incomplete proxy for how large language models behave inside real agent loops. We evaluate frontier and near-frontier LLMs in a timed multi-phase Risk environment with constrained output grammar, explicit victory targets, and repeated planning/execution cycles. In a replicated 32-game cross-provider championship under frozen rules, `gemini-3.1-pro-preview` was the strongest full-stack live agent, winning 20 of 32 games against `gpt-5.1`, `claude-opus-4-7`, and `kimi-k2.6`; the pooled winner distribution differs strongly from the equal-strength null ($p \approx 1.5 \times 10^{-5}$). However, the main applied result is not only a provider leaderboard. Once execution is standardized to a shared cheap Gemini Flash scaffold, the provider spread compresses sharply, and a pooled 32-game planner bakeoff is consistent with near-equality ($p \approx 0.821$). We then analyze saved planning and execution traces from the provider championship. Gemini's visible plans reference the terminal objective far more often than the others and scale that objective tracking as victory approaches. On execution, Gemini is not the cleanest runtime, but it converts more turns into deep conquest chains than the rest of the field. These findings suggest that live-agent performance depends on the interaction between objective tracking, execution conversion, cost, and runtime reliability, not just benchmark rank or release recency.

1 Introduction

Most public model comparisons still treat LLMs as static respondents: ask a question, score the answer, and rank the model. That is useful up to a point, but it is not how many real systems are built. In production, models operate inside bounded workflows with timers, format constraints, repeated phases, and failure modes. A model that looks excellent in a benchmark can still underperform badly once it has to act repeatedly inside a synchronous loop.

This paper studies that gap in a concrete setting: multi-player Risk. Risk is useful here not because it is a universal intelligence test, but because it is a compact live-agent benchmark. It combines long-horizon objectives, attack-chain planning, constrained action syntax, adversarial interaction, repeated decision-making, and clear win conditions. Those properties make it a good stress test for operationally realistic LLM behavior.

The central question is straightforward:

> What changes when we evaluate LLMs as live strategic agents instead of benchmark respondents?

This paper makes four concrete contributions:

1. It provides a replicated full-stack cross-provider live-agent benchmark under a fixed timed strategic protocol.
2. It shows that provider spread changes substantially once planning and execution are decomposed.
3. It provides capability anchors for `kimi-k2.6` against older closed-model tiers rather than only against the current frontier.
4. It supplements outcome tables with observable planning and execution trace analyses.

Three empirical findings stand out. First, provider differences are real in the full-stack live-agent setting: in our replicated 32-game provider championship, Gemini was the strongest tested full-stack model stack. Second, the strongest deployed agent is not necessarily the newest or most expensive monolithic model: in our OpenAI-only lineage tests, `gpt-5.1` outperformed newer OpenAI variants under the live-turn loop. Third, and most relevant for practitioners, system decomposition matters: when execution is standardized to a shared cheap Gemini Flash scaffold, the provider spread shrinks sharply. That suggests that much of what looks like a "model difference" in end-to-end play is really a system-design difference across planning, execution, timing, and cost.

The paper is therefore not just a leaderboard report. It is a live-agent systems study.

2 Positioning and Scope

This paper should not be read as a universal intelligence ranking. The object of study is narrower: **live-agent behavior under bounded execution constraints**.

The main claims are operational and comparative:

- which model stacks actually win under a timed multi-phase loop,
- how much of that result changes when planning and execution are separated, and
- what those changes imply for the design of practical LLM systems.

The study therefore sits between benchmark evaluation and systems evaluation. Risk is the domain, but the broader target is any workflow where models must repeatedly produce constrained actions under budget.

3 Related Work

This paper sits at the intersection of three literatures.

The first is the literature on broad LLM benchmarking. Canonical scorecards such as MMLU, BIG-bench, and HELM made model comparison more systematic and more public [4, 8, 5]. But those benchmarks mostly evaluate models as respondents rather than as agents embedded in repeated, timed, stateful loops. That gap matters because many deployed systems are limited by formatting, latency, tool use, or recovery from prior mistakes rather than by single-shot answer quality alone.

The second is the literature on interactive and agentic evaluation. Benchmarks such as AgentBench, GAIA, and τ-bench move closer to realistic usage by evaluating multi-step action, tool interaction, and environment feedback [6, 7, 9]. Our work is aligned with that shift. The difference is that the present study focuses on a synchronous adversarial environment with repeated strategic phases, explicit victory conditions, and hard turn budgets rather than open-ended assistant tasks or tool-use episodes.

The third is the literature on strategic reasoning in game-like settings. Gandhi et al. [3] study strategic reasoning with language models in stylized game-theoretic settings, while CICERO

demonstrated that language plus planning can reach human-level performance in Diplomacy [1]. More recently, GameBench has argued for game environments as a way to probe strategic reasoning more directly [2]. Our work is closest in spirit to this line, but differs in emphasis. We are not primarily proposing a new benchmark suite or a specialized game agent. We are measuring how commercially accessible frontier and near-frontier models behave inside a frozen, repeatedly executed, costed live-agent loop, and then asking what changes when the system is decomposed into planning and execution.

That combination is the main gap this paper addresses. Existing work gives us broad capability scorecards, agent benchmarks, and game-strategy benchmarks. What is still comparatively sparse is a replicated study of live multi-provider competition under fixed budgets that also tracks cost, hybridization, and trace-level mechanisms.

4 Why Risk Is a Useful Agent Benchmark

Risk has several properties that make it a useful strategic agent testbed:

1. It has a clear terminal objective. In our main runs, agents were explicitly instructed to win by reaching 65% territory control.
2. It requires multi-step local optimization. A good turn often involves a placement decision, a candidate attack chain, a decision to stop or continue, and a fortify choice.
3. It is adversarial and stateful. A strong move must account for board geometry, continents, troop counts, and opponent structure.
4. It exposes operational weaknesses. Because the game is broken into repeated timed phases with strict output grammar, timeout drag and fallback behavior become measurable rather than hypothetical.

That makes Risk well-suited to separating abstract "sounds strategic" behavior from actual goal-directed agent performance.

5 Experimental Harness

5.1 Environment

All experiments used the same underlying Risk engine, the same legality assistance from the engine, and the same output grammar. The main live strategic condition was:

- planning timer: 90 s
- execution turn timer: 90 s
- placement timer: 15 s
- primary victory condition: 65% territory control
- seat rotation enabled

The turn loop was decomposed into pre-turn planning, card trade, troop placement, repeated attacks, and fortify.

5.2 Core experiment inventory

Table 1: Core experiment inventory used in this manuscript.

Experiment	Sample	Question	Role in paper
Cross-provider championship	32 games	Which full-stack provider wins under the frozen strategic condition?	Main result
OpenAI generation ladder	32 games	Does newer OpenAI release status imply stronger live-agent play?	Recency / prestige result
Kimi anchors vs GPT-4.1 / Gemini 2.5 Pro / Sonnet 4	16 games each	Where does Kimi sit relative to older closed tiers?	Capability anchoring
Gemini Flash cost gate	15 games	Can a cheaper execution scaffold preserve enough strength?	Cost / system design
Flash-exec planner bakeoff	32 games	Do provider planning differences remain large after execution is standardized?	Decomposition result
Provider-32 trace analyses	946 turns	What planning and execution patterns explain the provider outcome?	Mechanism sections

5.3 Endpoints and statistics

The primary endpoint throughout the later experiment program was wins under the configured victory condition. Secondary endpoints included final territory totals, successful and failed attacks, attack-turn rate, fallback counts, invalid move rates, a strategic trace rubric, and estimated API cost where usage logging was available.

The primary omnibus comparison used winner-label permutation tests or equivalent equal-winner Monte Carlo checks depending on the analysis layer. Pairwise win reads used exact binomial logic conditioned on the relevant winner subset. This was supplemented by descriptive execution metrics and later by observable planning and execution trace analyses.

The goal here was not to manufacture artificial precision. It was to align the statistical read with the actual optimization target the agents were given: win the game.

5.4 Reproducibility and artifacts

Every major run in this paper is backed by saved experiment artifacts, tracked notes, and reusable analysis scripts. Runtime artifacts are stored under the shared `game_results` tree, while tracked interpretation lives in the repository documentation. The newer cost and trace analyses are script-backed rather than hand-tabulated.

6 Main Full-Stack Result: Gemini Wins the Provider Field

The strongest result in the entire program is the replicated cross-provider championship.

Two clean 16-game provider blocks were run under the same frozen strategic condition and then pooled:

- replicate 1: Gemini 10/16
- replicate 2: Gemini 10/16
- pooled result: Gemini 20/32, OpenAI 6/32, Claude 4/32, Kimi 2/32

The pooled winner vector differs strongly from the equal-strength null:

- omnibus equal-winner test: $p \approx 1.5 \times 10^{-5}$
- probability of a specified player winning at least 20/32 in a four-player equal-strength field: 7.98×10^{-6}

Conditioned pairwise winner splits also separate Gemini from each rival:

- Gemini vs GPT-5.1: 20–6, two-sided $p \approx 0.00936$
- Gemini vs Claude: 20–4, two-sided $p \approx 0.00154$
- Gemini vs Kimi: 20–2, two-sided $p \approx 0.000121$

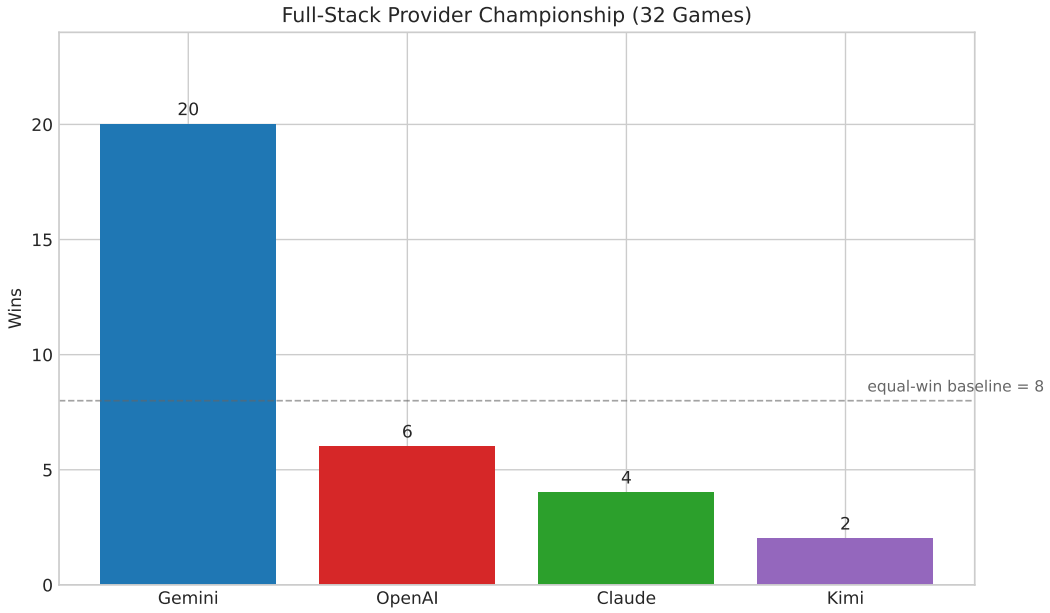

Figure 1: Full-stack provider championship. Pooled wins over 32 games under the frozen full-stack provider setup; Gemini is the only stack with a large replicated lead.

This is not a weak directional result. It is a stable replicated outcome. In this live-agent harness, `gemini-3.1-pro-preview` was the strongest tested full-stack provider representative. That does not mean Gemini is universally best, but it does mean Gemini is the strongest tested full-stack live agent in this specific bounded strategic environment.

7 Newest or Most Expensive Does Not Mean Best

The provider result is not the only place where benchmark prestige and deployment reality diverged.

In the pooled OpenAI generation ladder, `gpt-5.1` emerged as the strongest current OpenAI full-stack baseline in this environment, outperforming newer OpenAI variants in a live-turn condition. That is a meaningful systems result. It suggests that synchronous multi-phase agent loops reward a specific blend of speed, tactical conversion, and bounded reasoning discipline rather than raw flagship status.

The live loop punishes overlong planning, timeout-driven fallback, fragile formatting, and poor conversion between plan and attack sequence. That makes "best benchmark model" and "best live agent" different questions.

8 Kimi and the Benchmark Mirage

The Kimi experiments were designed to answer a different question: if Kimi is not clearly frontier-class in live play, where does it actually sit?

Three duplicate-team anchor arenas were run:

- 2×Kimi vs 2×GPT-4.1
- 2×Kimi vs 2×Gemini 2.5 Pro
- 2×Kimi vs 2×Claude Sonnet 4 (2025-05-14)

The pooled picture is coherent:

- near GPT-4.1
- somewhat below Gemini 2.5 Pro
- competitive with older Anthropic Sonnet 4 tier
- far below the current Gemini 3.1 frontier result

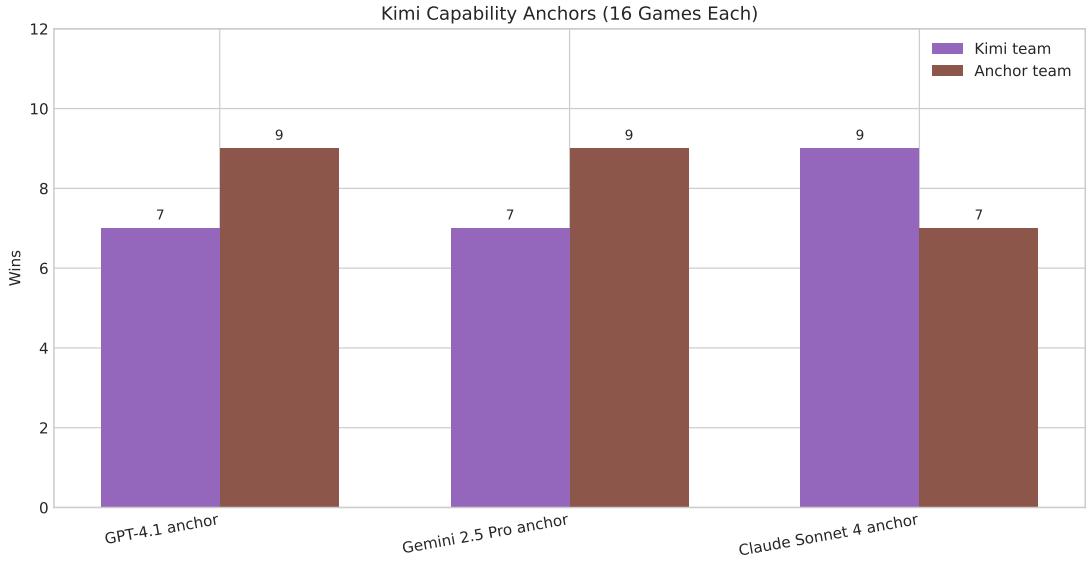

Figure 2: Kimi anchor experiments. Kimi is competitive with older strong closed-model tiers but clearly below the current Gemini 3.1 frontier result.

Google announced Gemini 2.5 Pro on March 25, 2025, with general availability on June 17, 2025. Moonshot announced Kimi K2.6 on April 21, 2026. So depending on which Gemini release milestone one uses, Kimi 2.6 arrives roughly 10–13 months later.

Yet in this live-agent environment, Kimi only reaches near-parity or modestly trails that older Gemini tier. That does not justify a literal calendar-lag theorem. But it does justify a strong practical interpretation: benchmark-near-parity narratives can substantially overstate real operational parity.

9 The System Design Turn: Decomposition Beats Monolithic Thinking

The most important result for builders came after the provider leaderboard question had largely been answered. We then asked a different question:

> Can we make the benchmark agent materially cheaper without giving away too much strength?

That led to a Gemini execution cost gate:

- `gemini-3.1-pro-full`
- `gemini-3-flash-full`
- `gemini-3.1-plan_gemini-3-flash-exec`

In the pooled 15-game cost gate:

- `gemini-3.1-plan_gemini-3-flash-exec`: 8/15 wins
- `gemini-3.1-pro-full`: 4/15
- `gemini-3-flash-full`: 3/15

Estimated total cost:

- `gemini-3.1-pro-full`: about $6.28
- `gemini-3.1-plan_gemini-3-flash-exec`: about $2.80
- `gemini-3-flash-full`: about $2.08

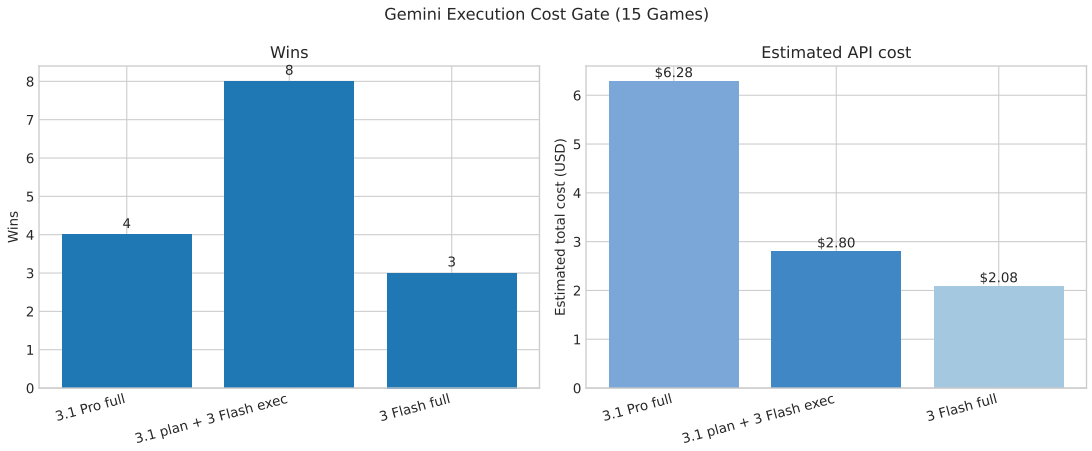

Figure 3: Gemini execution cost gate. The hybrid of Gemini 3.1 planning plus Gemini 3 Flash execution preserves most of the strength while cutting cost materially.

The best practical benchmark agent is therefore not necessarily the strongest single full-stack model. It may be a hybrid: stronger model for planning, cheaper faster model for execution.

10 Planner Rankings Shrink Once Execution Is Standardized

After locking a shared Flash execution scaffold, we ran a pooled 32-game planner bakeoff:

- Gemini planning + Flash execution
- GPT-5.5 planning + Flash execution
- Claude planning + Flash execution
- Kimi planning + Flash execution

The result was striking not because someone won decisively, but because almost nobody did.

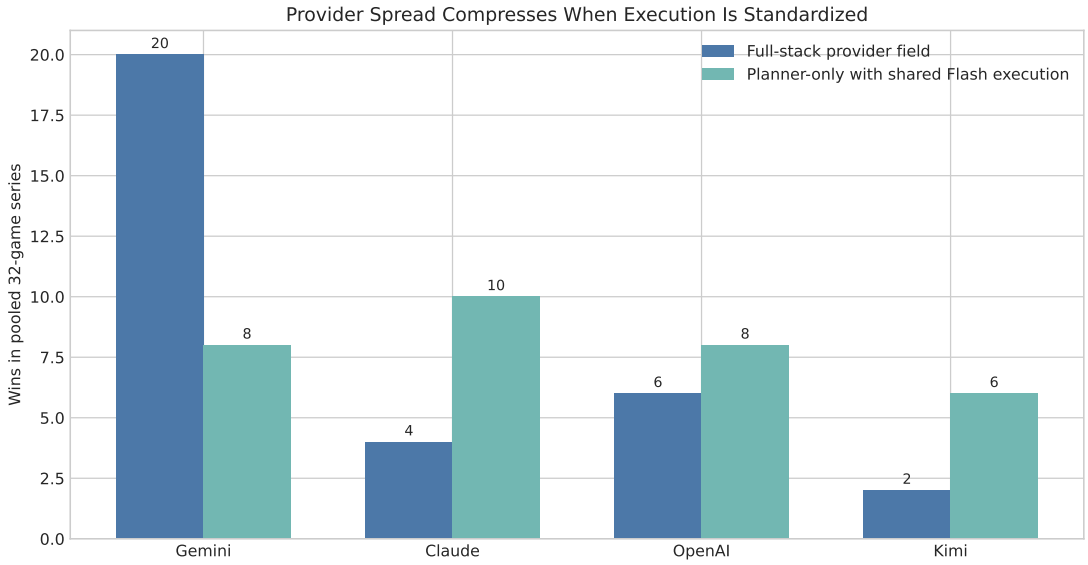

Figure 4: Full-stack spread versus planner-only spread. Once execution is standardized to Gemini Flash, the provider spread compresses sharply.

Once execution was fixed to the same cheap Gemini Flash layer, the provider spread compressed sharply. The pooled 32-game planner result was consistent with near-equality. Even a direct Claude-vs-Kimi duplicate-team duel stayed near parity at 9–7.

The statistical read is important:

- four-way pooled planner bakeoff omnibus: $p \approx 0.821$
- direct Claude-vs-Kimi duplicate-team duel: 9–7, two-sided $p \approx 0.804$

This has a strong design implication: much of the earlier provider spread was really about end-to-end system behavior. Planning differences exist, but they are much smaller than the earlier full-stack leaderboard implied.

11 Mechanism 1: Gemini Tracks the Objective More Explicitly

The next step was to stop looking only at outcomes and inspect the saved planning traces directly.

A new trace-analysis pass over the pooled provider-32 series looked at the observable `plan.text` from 946 saved turn summaries. The goal was to test a concrete hypothesis: does Gemini maintain the terminal objective more explicitly in its visible planning traces?

The answer is yes.

Plan share with explicit endgame-goal language:

- Gemini: 58.5%
- Claude: 3.1%
- Kimi: 1.4%
- GPT-5.1: 0.4%

Plan share with quantified goal language:

- Gemini: 54.5%
- Claude: 3.5%
- GPT-5.1: 0.8%
- Kimi: 0.5%

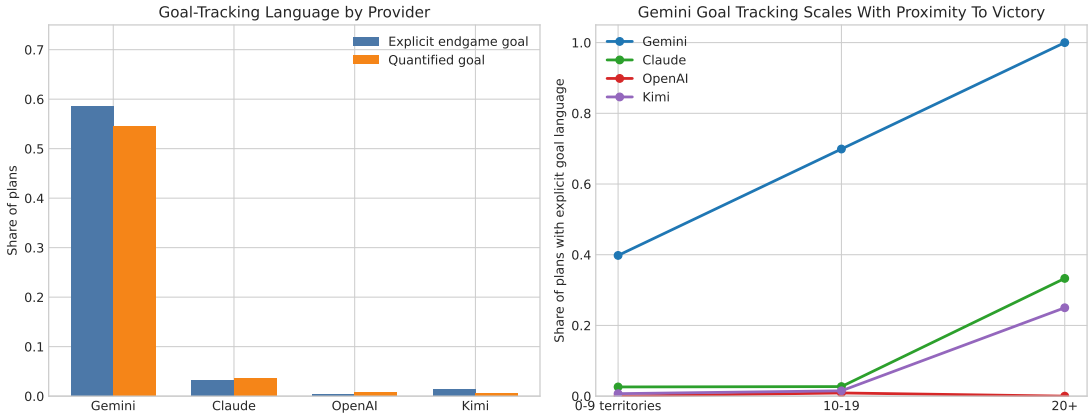

Figure 5: Goal-directedness trace analysis. Gemini references the terminal objective far more often than the other providers and increases that focus as it approaches victory.

This is not just a verbosity artifact. Even after normalizing by plan length, Gemini remains the clear outlier on explicit objective tracking.

Gemini's share of plans with explicit goal language rises from 39.8% when holding 0–9 territories, to 69.9% in the 10–19 band, to 100% in turns where it already controls 20+ territories. That is exactly what a goal-directed live agent should look like.

Across all providers pooled, turns with explicit endgame-goal language averaged 5.189 territories gained, compared with 4.080 for turns without such language. That does not establish causality, but it is directionally consistent with the mechanism hypothesis.

12 Mechanism 2: Gemini Converts Turns Better in Execution

Planning language alone is not enough. The model still has to turn a turn budget into board control.

The execution-trace analysis over the same pooled provider-32 series shows that Gemini is *not* the cleanest runtime. Claude is cleaner. GPT can be similarly aggressive. Kimi is sometimes

efficient. But Gemini converts live turns into deeper conquest chains more often than the rest of the field.

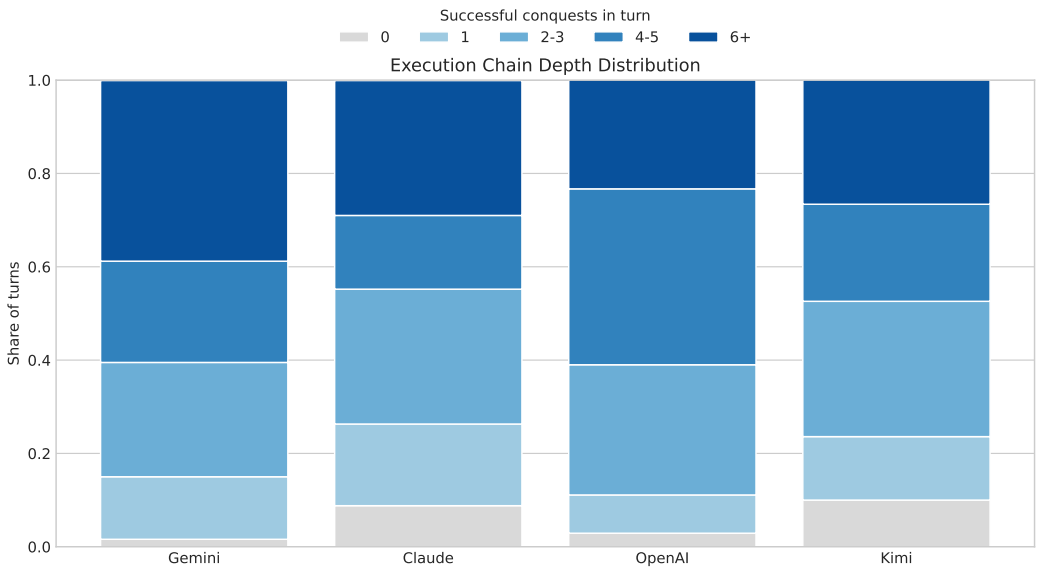

Figure 6: Execution chain depth distribution. Gemini produces deep conquest chains more often than the rest of the provider field.

Share of turns with 6+ successful conquests:

- Gemini: 38.7%
- Claude: 28.9%
- Kimi: 26.7%
- GPT-5.1: 23.4%

Gemini also had the best midgame territory conversion:

- Gemini: 5.363 territories gained per turn when starting with 10–19 territories
- Kimi: 4.500
- GPT-5.1: 4.150
- Claude: 4.068

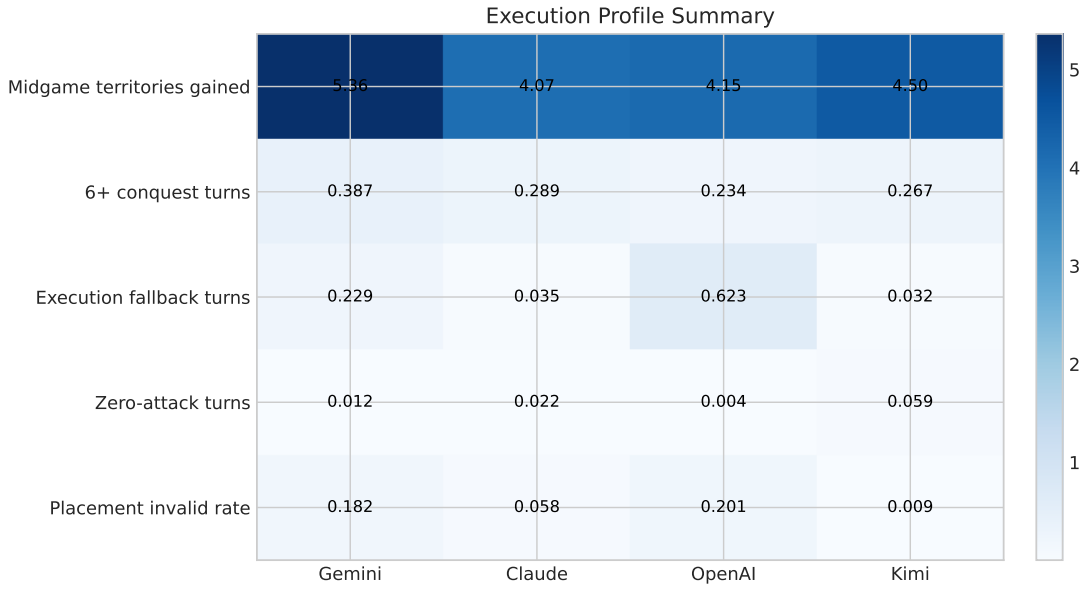

Figure 7: Execution profile summary. Gemini is not the cleanest runtime, but it combines acceptable reliability with the strongest midgame conversion.

This resolves a potentially misleading interpretation. Gemini did not win because everyone else was simply broken. Claude was cleaner. GPT could still attack aggressively. Kimi often stayed cheap and viable. Gemini won because it combined acceptable reliability with unusually strong conquest conversion.

13 Implications for Building LLM Systems

These experiments matter beyond Risk because they expose three general lessons for builders.

13.1 Evaluate models inside workflows, not just on benchmarks

A benchmark score does not tell you whether a model will survive a timed loop, maintain output format discipline, keep the real objective live, or convert a plan into repeated high-yield actions. If your system is multi-step, synchronous, or tool-using, those factors matter directly.

13.2 Separate planning and execution when it helps

The strongest practical scaffold in this project is not a monolithic flagship model. It is a hybrid: stronger Gemini planning plus cheaper Gemini Flash execution. This is likely a common pattern in real systems: expensive reasoning where it matters, cheaper execution where throughput and latency dominate.

13.3 Cost is part of capability

For research, it is tempting to treat cost as secondary. For deployed agent systems, that is a mistake. A model that is strong but too expensive or too slow to run repeatedly may not be the best system component. That is why this program explicitly tracked token usage, fallback behavior, and estimated cost where possible. Those are part of the engineering question.

14 Limitations

This study has clear limitations:

1. It is one game domain.
2. It uses one prompt grammar family and one legality-assistance strategy.
3. It uses one main timer regime.
4. The planning-trace analysis is observational. It studies visible text, not hidden reasoning.
5. The "months behind" interpretation for Kimi is anchor-based and approximate, not a formal time-indexed theorem.
6. Some planner-only comparisons remain inconclusive, especially once execution is standardized.

These limitations should narrow the interpretation, not erase the result. The correct claim is not "we have solved model ranking." The correct claim is that live-agent evaluation produces materially different conclusions than static benchmark culture would suggest.

15 Conclusion

The main conclusion of this work is simple:

> Live-agent quality is a systems property, not a benchmark rank.

In this Risk-based strategic benchmark:

- Gemini was the strongest replicated full-stack provider representative.
- OpenAI's best live-turn baseline was not its newest flagship-style variant.
- Kimi was materially behind the current frontier despite strong benchmark-adjacent expectations.
- A cheap planning/execution hybrid produced a better practical benchmark scaffold.
- Once execution was standardized, much of the provider spread in planning collapsed.

That is the real contribution of this study. If we want to understand how LLMs behave in real agent systems, we have to stop treating them like isolated benchmark respondents and start treating them like components in bounded workflows.

16 Established Claims

Claims strongly supported by the current evidence:

- Gemini is the strongest tested full-stack provider representative in this harness.
- Benchmark-near-parity claims can materially overstate live operational parity.
- Cost-aware planning/execution decomposition can improve practical deployment value.
- Standardizing execution compresses much of the apparent provider spread.

Claims that should remain softer:

- universal planner rankings
- exact provider "months behind" statements

- broad generalization to all agent domains
- causal proof that goal-directed language directly causes better play

A Appendix: Core Experiment Tables

Table 2: Main full-stack provider result used throughout the manuscript.

Provider stack	Wins	Win rate
Gemini 3.1 Pro	20	62.5%
OpenAI representative	6	18.75%
Claude Opus 4.7	4	12.5%
Kimi K2.6	2	6.25%

Table 3: Planner-only bakeoff with execution standardized to Gemini 3 Flash.

Planner stack	Wins	Win rate
Claude Opus 4.7 + Gemini 3 Flash exec	10	31.25%
Gemini 3.1 + Gemini 3 Flash exec	8	25.0%
GPT-5.5 + Gemini 3 Flash exec	8	25.0%
Kimi K2.6 + Gemini 3 Flash exec	6	18.75%

Table 4: Gemini execution cost gate.

Configuration	Wins	Estimated total cost	Cost per win
Gemini 3.1 Pro full	4	$6.28	$1.57
Gemini 3.1 plan + Gemini 3 Flash exec	8	$2.80	$0.35
Gemini 3 Flash full	3	$2.08	$0.69

B Appendix: Reproducibility and Artifact Links

Artifact category	Location
Repository	https://github.com/hcekne/risk-game
Tracked experiment suite index	`docs/experiment-suites/2026-q2-strategic-tests/README.md`
Main provider pooled note	`docs/experiment-suites/2026-q2-strategic-tests/2026-05-07_cross-provider-frontier-strategic-32-pooled.md`
Goal-directedness trace note	`docs/experiment-suites/2026-q2-strategic-tests/2026-05-10_provider32-goal-directedness-trace-analysis.md`
Execution trace note	`docs/experiment-suites/2026-q2-strategic-tests/2026-05-10_provider32-execution-trace-analysis.md`

Artifact category	Location
Gemini Flash cost gate note	`docs/experiment-suites/2026-q2-strategic-tests/2026-05-09_gemini3-flash-execution-cost-gate.md`
Figure generation script	`scripts/build_live_agent_risk_article_figures.py`
PDF manuscript build script	`scripts/build_live_agent_risk_latex.sh`
OCR sanity check script	`scripts/ocr_check_live_agent_risk_pdf.sh`
Shared runtime artifact root	`/shared-game-results` inside the container workflow

References

[1] Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, et al. Human-level play in the game of diplomacy by combining language models with strategic reasoning. *Science*, 378(6624):1067–1074, 2022. doi: 10.1126/science.ade9097.

[2] Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng, Wenjie Li, Joshua Clymer, and Arjun Yadav. Gamebench: Evaluating strategic reasoning abilities of llm agents. *arXiv preprint arXiv:2406.06613*, 2024. URL https://arxiv.org/abs/2406.06613.

[3] Kanishk Gandhi, Dorsa Sadigh, and Noah D. Goodman. Strategic reasoning with language models. *arXiv preprint arXiv:2305.19165*, 2023. URL https://arxiv.org/abs/2305.19165.

[4] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. URL https://arxiv.org/abs/2009.03300.

[5] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022. URL https://arxiv.org/abs/2211.09110.

[6] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023. URL https://arxiv.org/abs/2308.03688.

[7] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983*, 2023. URL https://arxiv.org/abs/2311.12983.

[8] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*, 2022. URL https://arxiv.org/abs/2206.04615.

[9] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024. URL https://arxiv.org/abs/2406.12045.